Credit Card Fraud Detection using Scikit-Learn

In this project you will consolidate your machine learning (ML) modeling skills by using two popular classification models to recognize fraudulent credit card transactions. These models are: Decision Tree and Support Vector Machine. You will use a real dataset to train each of these models. The dataset includes information about transactions made by credit cards in September 2013 by European cardholders. You will use the trained model to assess if a credit card transaction is legitimate or not.

In [6]:
```
# install the opendatasets package
!pip install opendatasets

import opendatasets as od

# download the dataset (this is a Kaggle dataset)
# during download you will be required to input your Kaggle username and pa
ssword
od.download("https://www.kaggle.com/mlg-ulb/creditcardfraud")
```

```
Requirement already satisfied: opendatasets in c:\users\ben ashael\anaconda
3\lib\site-packages (0.1.22)
Requirement already satisfied: kaggle in c:\users\ben ashael\anaconda3\lib
\site-packages (from opendatasets) (1.5.12)
Requirement already satisfied: click in c:\users\ben ashael\anaconda3\lib\s
ite-packages (from opendatasets) (7.0)
Requirement already satisfied: tqdm in c:\users\ben ashael\anaconda3\lib\si
te-packages (from opendatasets) (4.31.1)
Requirement already satisfied: urllib3 in c:\users\ben ashael\anaconda3\lib
\site-packages (from kaggle->opendatasets) (1.24.1)
Requirement already satisfied: certifi in c:\users\ben ashael\anaconda3\lib
\site-packages (from kaggle->opendatasets) (2019.3.9)
Requirement already satisfied: python-dateutil in c:\users\ben ashael\anaco
nda3\lib\site-packages (from kaggle->opendatasets) (2.8.0)
Requirement already satisfied: requests in c:\users\ben ashael\anaconda3\li
b\site-packages (from kaggle->opendatasets) (2.21.0)
Requirement already satisfied: six>=1.10 in c:\users\ben ashael\anaconda3\l
ib\site-packages (from kaggle->opendatasets) (1.12.0)
Requirement already satisfied: python-slugify in c:\users\ben ashael\anacon
da3\lib\site-packages (from kaggle->opendatasets) (8.0.1)
Requirement already satisfied: idna<2.9,>=2.5 in c:\users\ben ashael\anacon
da3\lib\site-packages (from requests->kaggle->opendatasets) (2.8)
Requirement already satisfied: chardet<3.1.0,>=3.0.2 in c:\users\ben ashael
\anaconda3\lib\site-packages (from requests->kaggle->opendatasets) (3.0.4)
Requirement already satisfied: text-unidecode>=1.3 in c:\users\ben ashael\a
naconda3\lib\site-packages (from python-slugify->kaggle->opendatasets) (1.
3)
Skipping, found downloaded files in ".\creditcardfraud" (use force=True to
force download)
```

In [7]:
```
# Snap ML is available on PyPI. To install it simply run the pip command be
low.
!pip install snapml
```

```
Requirement already satisfied: snapml in c:\users\ben ashael\anaconda3\lib
\site-packages (1.12.0)
Requirement already satisfied: scipy in c:\users\ben ashael\anaconda3\lib\s
ite-packages (from snapml) (1.2.1)
Requirement already satisfied: scikit-learn in c:\users\ben ashael\anaconda
3\lib\site-packages (from snapml) (0.20.3)
Requirement already satisfied: numpy>=1.18.5; python_version < "3.10" in
c:\users\ben ashael\anaconda3\lib\site-packages (from snapml) (1.21.6)
```

Import Libraries

In [8]:
```
# Import the libraries we need to use in this lab
from __future__ import print_function
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import normalize, StandardScaler
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.metrics import roc_auc_score
import time
import warnings
warnings.filterwarnings('ignore')
```

Dataset Analysis

In [9]:
```
# read the input data
raw_data = pd.read_csv('creditcardfraud/creditcard.csv')
print("There are " + str(len(raw_data)) + " observations in the credit card
fraud dataset.")
print("There are " + str(len(raw_data.columns)) + " variables in the datase
t.")

# display the first rows in the dataset
raw_data.head()
```

```
There are 284807 observations in the credit card fraud dataset.
There are 31 variables in the dataset.
```

Out[9]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	

5 rows × 31 columns

In [10]:
```
n_replicas = 10

# inflate the original dataset
big_raw_data = pd.DataFrame(np.repeat(raw_data.values, n_replicas, axis=0),
columns=raw_data.columns)

print("There are " + str(len(big_raw_data)) + " observations in the inflate
d credit card fraud dataset.")
print("There are " + str(len(big_raw_data.columns)) + " variables in the da
taset.")

# display first rows in the new dataset
big_raw_data.head()
```

```
There are 2848070 observations in the inflated credit card fraud dataset.
There are 31 variables in the dataset.
```

Out[10]:

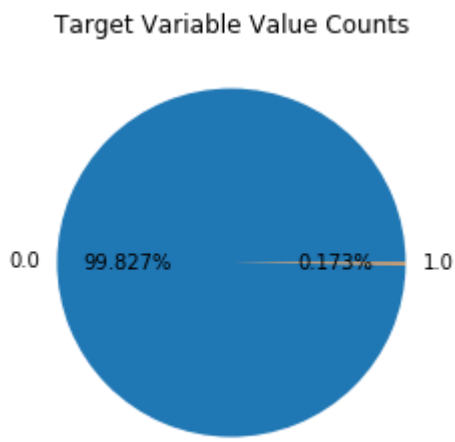
	Time	V1	V2	V3	V4	V5	V6	V7	V8	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.36
1	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.36
2	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.36
3	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.36
4	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.36

5 rows × 31 columns

In [11]:
```
# get the set of distinct classes
labels = big_raw_data.Class.unique()

# get the count of each class
sizes = big_raw_data.Class.value_counts().values

# plot the class value counts
fig, ax = plt.subplots()
ax.pie(sizes, labels=labels, autopct='%1.3f%%')
ax.set_title('Target Variable Value Counts')
plt.show()
```



Dataset Preprocessing

In [15]:
```
# data preprocessing such as scaling/normalization is typically useful for
# linear models to accelerate the training convergence

# standardize features by removing the mean and scaling to unit variance
big_raw_data.iloc[:, 1:30] = StandardScaler().fit_transform(big_raw_data.il
oc[:, 1:30])
data_matrix = big_raw_data.values

# X: feature matrix (for this analysis, we exclude the Time variable from t
he dataset)
X = data_matrix[:, 1:30]

# y: labels vector
y = data_matrix[:, 30]

# data normalization
X = normalize(X, norm="l1")

# print the shape of the features matrix and the labels vector
print('X.shape=', X.shape, 'y.shape=', y.shape)
```

```
X.shape= (2848070, 29) y.shape= (2848070,)
```

Dataset Train/Test Split

In [16]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ra
ndom_state=42, stratify=y)
print('X_train.shape=', X_train.shape, 'Y_train.shape=', y_train.shape)
print('X_test.shape=', X_test.shape, 'Y_test.shape=', y_test.shape)
```

```
X_train.shape= (1993649, 29) Y_train.shape= (1993649,)
X_test.shape= (854421, 29) Y_test.shape= (854421,)
```

Build a Decision Tree Classifier model with Scikit-Learn

In [19]:
```
# compute the sample weights to be used as input to the train routine so th
at
# it takes into account the class imbalance present in this dataset
w_train = compute_sample_weight('balanced', y_train)

# import the Decision Tree Classifier Model from scikit-learn
from sklearn.tree import DecisionTreeClassifier

# for reproducible output across multiple function calls, set random_state
to a given integer value
sklearn_dt = DecisionTreeClassifier(max_depth=4, random_state=35)

# train a Decision Tree Classifier using scikit-learn
t0 = time.time()
sklearn_dt.fit(X_train, y_train, sample_weight=w_train)
sklearn_time = time.time()-t0
print("[Scikit-Learn] Training time (s):  {0:.5f}".format(sklearn_time))
```

```
[Scikit-Learn] Training time (s):  119.22944
```

Build a Support Vector Machine model with Scikit-Learn

In [21]:
```
# import the linear Support Vector Machine (SVM) model from Scikit-Learn
from sklearn.svm import LinearSVC

# instatiate a scikit-learn SVM model
# to indicate the class imbalance at fit time, set class_weight='balanced'
# for reproducible output across multiple function calls, set random_state
to a given integer value
sklearn_svm = LinearSVC(class_weight='balanced', random_state=31, loss="hin
ge", fit_intercept=False)

# train a linear Support Vector Machine model using Scikit-Learn
t0 = time.time()
sklearn_svm.fit(X_train, y_train)
sklearn_time = time.time() - t0
print("[Scikit-Learn] Training time (s):  {0:.2f}".format(sklearn_time))
```

```
[Scikit-Learn] Training time (s):  416.62
```